Stockbot | 주식 데이터를 반복해서 조사하기 위한 배치·웹 시스템

"어린이날을 앞두고 특정 종목이 자주 오를까?"라는 질문에서 시작했습니다. 기억에 남는 차트 몇 장 대신 여러 해의 가격과 거래량을 같은 기준으로 비교하고 싶었습니다. 국내 종목의 일별 시세를 수집하고, 가격 변화와 기간 통계를 계산해 웹에서 조회하는 개인용 리서치 도구를 만들었습니다.

2022~2025년 Rust 주 구현에서 수집·배치·PostgreSQL·Actix API·Yew/WASM 웹과 AWS 배포를 직접 구성했습니다. 초기 Go 프로토타입과 별도 Spring 구현 실험도 진행했습니다. 이들을 차례로 완전히 교체한 하나의 제품으로 설명하지는 않습니다. 2026년 AI 에이전트 보조 현대화는 아래 원래 개발과 구분합니다.

흐름은 **외부 시세 → 응답 파일 → 파싱·저장 → 변화율·방향 구간·기간 집계 → API → 웹 조회**입니다. 자동 매매 기능이나 투자 수익 검증은 이 프로젝트의 결과가 아닙니다.

1. 매번 전체 이력을 다시 받지 않으려면

종목마다 수집이 끝난 날짜가 달랐습니다. 전체 작업의 마지막 실행일만 믿으면 중간에 실패한 종목의 누락을 놓칠 수 있어, DB에서 종목별 최종 저장일을 읽고 이후 범위를 요청했습니다. 종목과 날짜를 묶은 키로 upsert해 같은 날짜를 다시 받아도 기존 행을 갱신하도록 했습니다.

다운로드·파싱·저장을 나눠 외부 응답 형식의 변경이 내부 분석까지 번지지 않게 했습니다. 2025년에는 이 경계를 유지하며 KRX 가격 수집 경로를 Naver 시세 경로로 바꿨습니다. 재실행 가능한 수집 구조를 만들었지만, 과거 시세 정정의 전면 추적까지 구현한 것은 아닙니다. Naver 다운로드에는 종목별 순차 요청도 남아 있습니다.

2. 긴 차트를 비교 가능한 구간으로 만들기

달력 날짜 차이에는 주말과 휴장일이 포함됩니다. 실제 거래가 있었던 날짜에 순번을 붙여 가격 변화의 길이를 거래일 기준으로 계산했습니다. 인접한 가격의 방향이 같은 동안 하나로 묶고, 방향이 바뀌면 새 구간을 만드는 로직을 작성했습니다. 이 묶음을 프로젝트 안에서 **bound**라고 불렀습니다.

예를 들어 종가 **100 → 110 → 120 → 115**는 상승 구간과 하락 구간으로 나뉩니다. 시가·고가·저가·종가를 각각 처리하고 시작·종료 가격, 거래일 길이, 변화율을 저장해 기간 집계와 조건 검색에 사용했습니다. 두 끝점의 **y = mx + b**도 계산했지만 이는 구간을 수치로 표현한 값이며 예측 모델은 아닙니다.

전체 입력을 요약하는 경로는 구현했지만, 증분 처리에서 기존 마지막 구간과 새 구간을 이어 붙이는 처리는 미완료입니다. 같은 방향이 계속되면 구간이 나뉘거나 겹칠 수 있어 마지막 기존 구간까지 함께 재계산하고 교체해야 합니다.

3. 긴 배치에서 어디까지 실행됐는지 알기

종목 정보, 가격, 변화율과 집계를 한 함수로 묶으면 실패 위치와 재실행 범위를 알기 어려웠습니다. Spring Batch의 구분을 참고해 전체 실행인 Batch, 업무 묶음인 Job, 처리 단계인 Step을 Rust 인터페이스로 나누고 시작·종료 시각과 실행 계획을 DB에 남겼습니다. 스케줄러와 제어 API는 한국 시간의 평일 실행과 배치용 EC2의 시작·종료 요청을 맡았습니다.

업무 단계와 실행 이력을 분리했지만 원래 오류 경로에는 종료 기록이 누락되는 결함이 있었습니다. 2026년 공개 사례집에서 실패가 Step·Job·Batch 종료로 이어지는 독립 예제를 만들고 시험했습니다. 이 결과를 원본 전체에 적용된 완성형 배치 프레임워크의 성과로 쓰지는 않습니다.

4. 동시 작업과 대량 적재를 같은 방식으로 풀 수 없었다

작업 수를 늘리자 DB 연결 대기와 pool timeout이 발생했습니다. 일부 경로에 용량 제한 채널을 적용해 다운로드·파싱·저장을 분리하고 작업자 수와 연결 풀을 함께 조정했습니다. 다만 전체 작업의 동시성 상한을 통일하지 못했고, 행 단위 upsert도 남았습니다.

대량 이력을 하나의 거대한 INSERT로 만들 때는 쿼리 크기와 bind 처리 문제가 생겼습니다. 소량 갱신에는 upsert, 묶음 처리에는 배열과 UNNEST, 전체 이력 적재에는 CSV와 COPY를 구현·실험했습니다. PostgreSQL에는 날짜 기준 연도별 RANGE 파티션과 사전 집계도 구성했습니다. 원래 파티션 DDL의 연말 경계 결함과 일부 조회의 파티션 활용 한계는 남았습니다.

2025년 보존 기록에는 원본 가격 약 1,104만 행과 파생 방향 구간 약 2,392만 행이 있습니다. 저장 데이터의 규모이며 실시간 처리량이나 이용자 수가 아닙니다.

5. 배치·API·웹과 실제 배포 환경을 연결하기

배치와 웹에서 가격 종류나 방향의 의미가 달라지지 않도록 Rust 공유 모델을 사용했습니다. 서버 전용 DB 의존성과 브라우저용 기능은 feature로 나눴고, 표 응답용 trait와 derive macro를 **tb_row** crate로 배포했습니다.

ARM64 서버 배포에서는 실행 파일 아키텍처와 OpenSSL·런타임 라이브러리가 맞지 않는 오류를 겪었습니다. Docker Buildx·QEMU와 ARM64 Ubuntu 이미지를 조합하고 GitHub Actions에서 ECR로 이미지를 전송했습니다. 웹은 S3, API와 스케줄러는 S3 배포 번들과 CodeDeploy를 연결했습니다. 빌드·배포 자동화이며 모든 테스트를 강제하는 배포 체계는 아니었습니다.

2026년에는 AI 에이전트 보조로 Axum·React·TypeScript 전환과 TimescaleDB 압축을 진행했습니다. 저장 공간은 줄었지만 압축된 과거 데이터와 기존 전체 갱신 배치의 호환성은 남았습니다. 최신 변경을 과거 단독 개발로 소급하거나 현재 운영이 모두 검증됐다고 주장하지 않습니다.

확인 자료

공개 기술 사례와 독립 실행 예제 · tb_row 패키지 · Actix examples 병합 기여

비공개 원본과 시장 원자료는 포함하지 않았습니다. 공개 사례집은 원본의 설계·결함과 재구성 예제의 검증 범위를 구분해 제공합니다.